

E-Puck Autonomous Parking

Reinforcement Learning with PPO in Webots

Introdução à Robótica Inteligente

Hugo Sousa - Up202305900

Tiago Silva - Up202307202

ZhiXu Ni - Up202205842

Problem Statement

The Challenge

Agent's objective:

- Navigate in a dynamic environment
- Avoid obstacles and other EPUCK robots
- Reach a free parking bay
- Minimize collisions and execution time

The Environment

Webots simulation with LiDAR, GPS and compass. Bay has narrow entrance with two walls either side.

The Approach

Train a PPO, SAC, TD3 agent (Stable-Baselines3) to learn steering from scratch through trial and error.

Problem & Motivation

Partially Known Environments

Classic algorithms (Dijkstra, A*) assume a fully known, static map. Real robots face unknown layouts and dynamically occupied spaces.

Combined Navigation + Parking



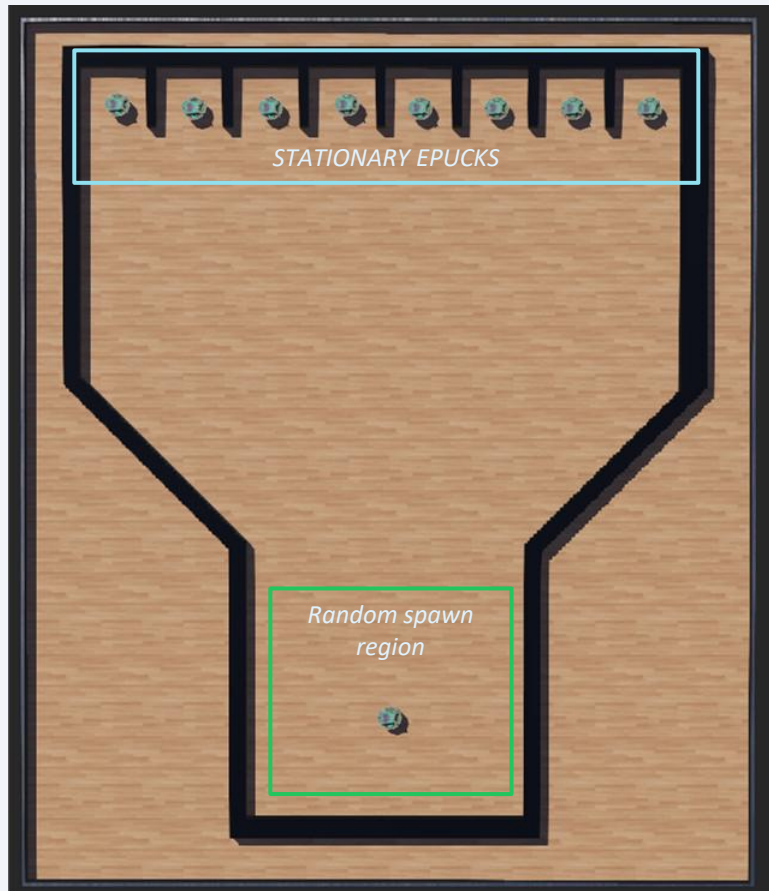
The robot must not only reach the vicinity of the bay — it must stop precisely inside it. Most prior RL work omits this final step.

Generalization via Randomization

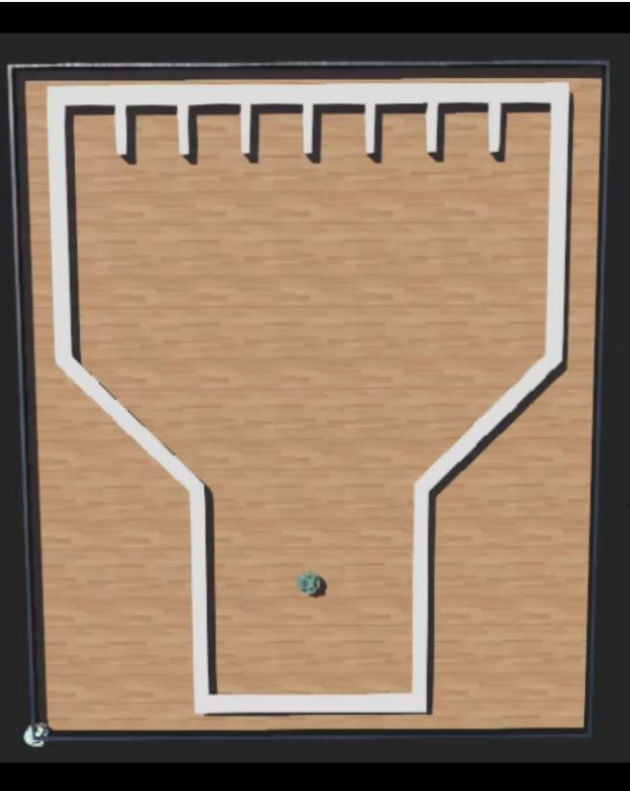


Free bays and starting position change every episode, forcing the policy to generalize rather than memorize a fixed route.

Training Environment



- 8 bays total — 6 occupied (obstacles), 2 free (target candidates) at random;
- Robot assigned 1 of the 2 free bays as its target each episode;
- Episode ends when one of the following occurs:
 - When Target is reached
 - Collision
 - Time out



D* Lite is an incremental path-planning algorithm that finds the shortest path in an initially unknown map while the robot moves, quickly replanning whenever new obstacles are discovered.

https://drive.google.com/file/d/1khTstL9KdttHuBEOJwoRbUX65PtEHls6/view?usp=drive_link

Reward Function Design

Collision

-100

Episode ends immediately

Success

+100

Reached target bay

Progress ×8

dense

$\Delta\text{dist} \cdot 8$ each step

Regression ×6

extra

$-6 \cdot |\Delta\text{dist}|$ if moving away

Orientation -1.5

angle

Penalize facing away from target

Anti-Stuck -40

stuck

>8 steps no progress → -40
penalty

Per-step -0.01

speed

Small constant penalty for delay

Reward Function — Implementation

```
def _reward(self, obs):  
    dist = self._distance()  
    if self.touch_sensor.getValue() == 1.0:  
        return -100.0, True    # collision  
    if dist < DIST_THRESHOLD:  
        return 100.0, True    # success  
    progress = self.prev_distance - dist  
    reward = 8.0 * progress    # main signal  
    if progress < 0:  
        reward += 6.0 * progress    # extra retreat penalty  
    angle_error = |desired_angle - robot_angle|  
    reward -= 1.5 * angle_error    # orientation penalty  
    reward -= 0.01                # speed incentive  
    if same_spot_steps > 8:    reward -= 40    # anti-stuck
```

Algorithms — Setup & Hyperparameters

PPO

Action space: Discrete

Network: 2×64 units

- On-policy
- 3 Action-space:
Move fwd / rotate L / rot R
- Clipped objective, stable progress
- Very stable

SAC

Action space: Continuous

Network: 2×256 units

- Off-policy
- Angular velocity control, fixed forward speed
- Entropy maximization, faster learning rate
- Stable

TD3

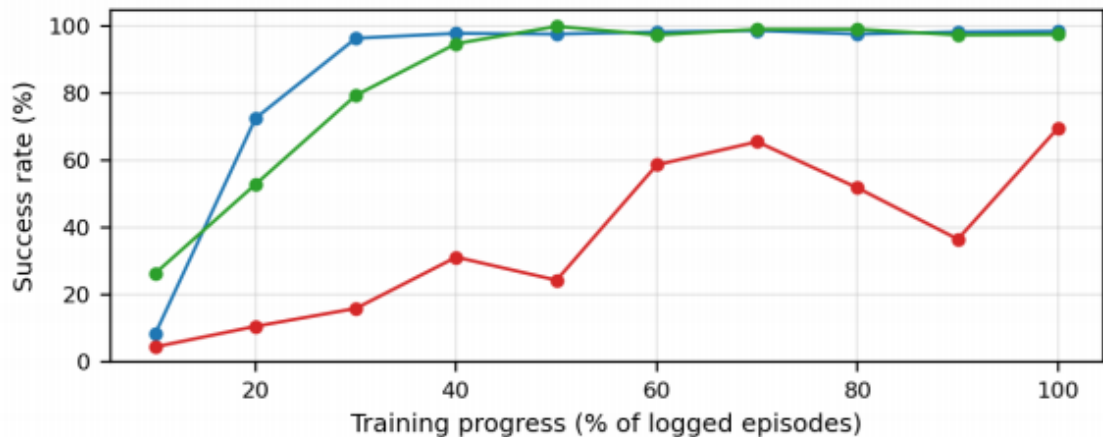
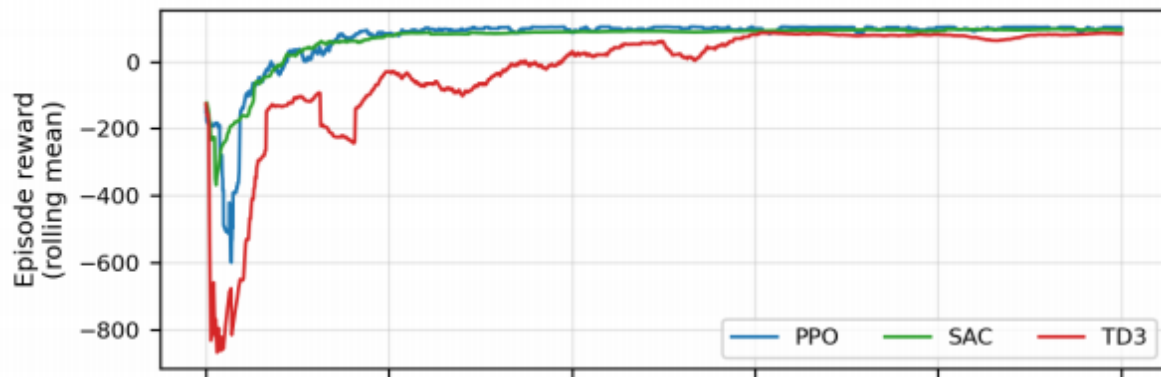
Action space: Continuous

Network: 2×256 units

- Off-policy
- Angular velocity control, fixed forward speed
- Delayed policy updates, slower learning rate
- High variance, noisy

All algorithms: $lr = 3 \times 10^{-4}$ · $\gamma = 0.99$ · Budget = 900 000 environment timesteps

Training Curves — Episode Reward & Success Rate



Results — Comparison on Best Checkpoint & Overall Performance

Table I — Best Evaluation Checkpoint

Highlights training metrics at different timesteps, revealing that PPO achieved the highest mean reward of 105.31 after 160,000 steps. SAC proved to be highly efficient by reaching a strong reward of 98.20 in just 80,000 steps, whereas TD3 required 140,000 steps to reach a much lower reward of 87.50.

TABLE I
BEST EVALUATION CHECKPOINT PER ALGORITHM

Algorithm	Timestep	Mean reward	Std	Mean ep. length
PPO	160,000	105.31	1.24	88.0
SAC	80,000	98.20	3.14	106.4
TD3	140,000	87.50	3.54	108.6

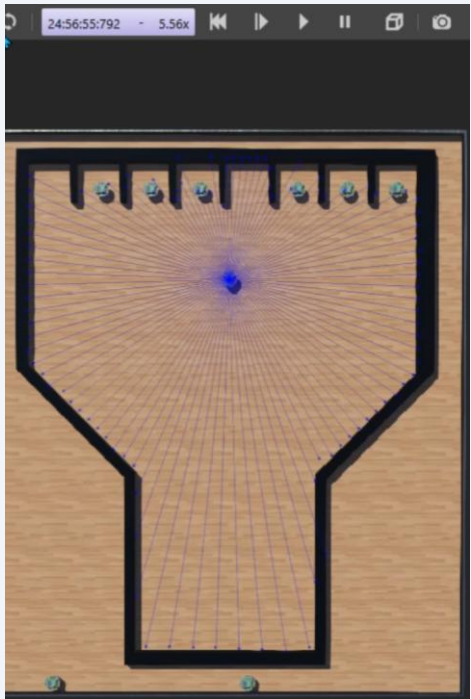
Table II — Full Training Performance

Compares three algorithms across different action spaces, showing that PPO achieved the highest success rate (98.4%) and mean reward over 4,832 episodes. In contrast, while SAC maintained a competitive 97.5% success rate, TD3 struggled significantly, yielding a success rate of just 53.4% and the longest median episode duration.

TABLE II
PERFORMANCE COMPARISON OVER THE ALL OF LOGGED EPISODES

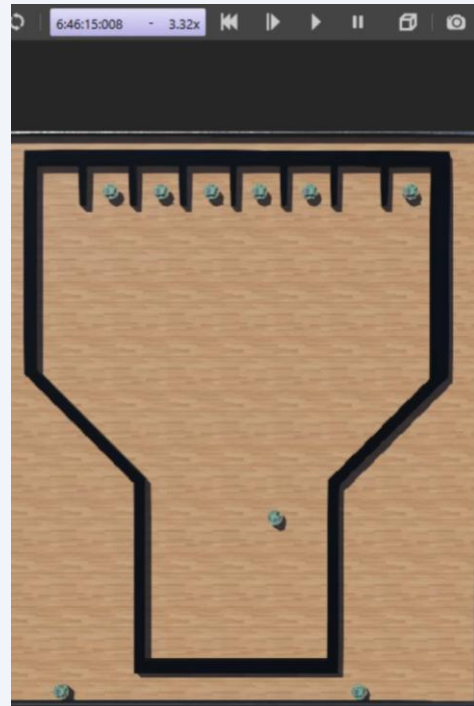
Metric	PPO	SAC	TD3
Action space	Discrete	Continuous	Continuous
Episodes logged	4832	1178	1318
Success rate (%)	98.4	97.5	53.4
Mean reward	101.3	96.7	78.3
Median episode length (steps)	84	105	107

Experiment PPO/SAC



https://drive.google.com/file/d/1WzK7hQK9oQhDvyp63w6J2Vryn7ozlMLA/view?usp=drive_link

Experiment TD3



https://drive.google.com/file/d/1zotP_VqXXOIdgg5GeHdN0dAnzIRLDtM4/view?usp=drive_link

Conclusions & Future Work

PPO and SAC achieve >97% success rate — RL is viable for combined navigation + parking in partially known environments.

SAC converges fastest (~60k steps); PPO reaches the highest final reward (105.31) with shortest episode length (84 steps).

TD3 lags significantly (~53% success) — likely due to continuous action space, hyperparameters, conservative deterministic exploration or reward function interactions with TD3 mechanics outweigh the stability twin critics and delayed updates.

Future Work

- Full parking criterion: require correct final orientation + minimum dwell time
- Refine reward to reduce stuck episodes (sub-goals, repeated-action penalties, improve/explore TD3's performance)
- Equal-timestep comparison to fairly isolate algorithm differences
- Sim-to-real transfer on a physical E-Puck with multiple robots / moving obstacles